源码导读 · 功能指南 · OMACOM/OMARCHY

Omarchy 4.0：从源码看懂这个系统，再把它用好

Omarchy 是 DHH 做的一套 Arch Linux + Hyprland 发行版层，4.0（quattro 分支）把它改成了 pacman 包发布、Lua 写的 Hyprland 配置和一个常驻的 Quickshell 桌面。这份导读分两半：前半从源码讲它怎么装、命令怎么路由、配置怎么分层、主题怎么一键换遍全系统、更新和迁移怎么跑、“agentic”具体指什么；后半是面向日常使用的功能指南：快捷键体系、菜单、剪贴板、听写、截屏录屏、AI 工具链、Web App、自定义与维护，以及哪些操作会覆盖你的配置。所有源码链接固定在同一个 commit，构建时逐条校验。

`omacom/omarchy @ e30e18c`

---

0. 怎么读这份导读

写给两类读者：想**每天把 Omarchy 用顺手**的人，和想**改它、给它提 PR** 的工程师。前一类可以直接跳到第 9、10 章（功能指南、定制与维护），后一类按顺序读。

1. 全景：它解决什么问题，4.0 的整体结构
2. 打包与安装：两个 Arch 包、三层家目录
3. CLI 路由器：`omarchy <group> <cmd>` 怎么找到脚本
4. Hyprland 的 Lua 配置与快捷键体系
5. Quickshell 桌面与菜单
6. 主题流水线：一条命令重新着色二十多个应用
7. 更新、迁移与快照
8. “Agentic”：AI 代理是怎么接进系统的
9. 功能指南：用好这个 OS
10. 定制与维护：改哪里、哪些操作会覆盖你的配置
11. 端到端：按下 `Super+Ctrl+T` 到全系统换色
12. 许可证 · 13. 贡献与定制 · 14. 源码索引

> **要点 · 固定版本**
>
> 所有源码链接都固定在 `omacom/omarchy` 的 `e30e18c`（2026-11-11，默认分支 `quattro`，`version` 文件为 `4.0.0.alpha`）。4.0 与 v1–v3 有根本区别：v3 是 `~/.local/share/omarchy` 下的 git 克隆、靠 `git pull` 更新；4.0 以 pacman 包发布，运行时路径是 `/usr/share/omarchy`。网上很多 v3 时代的教程在 4.0 上已经不适用。

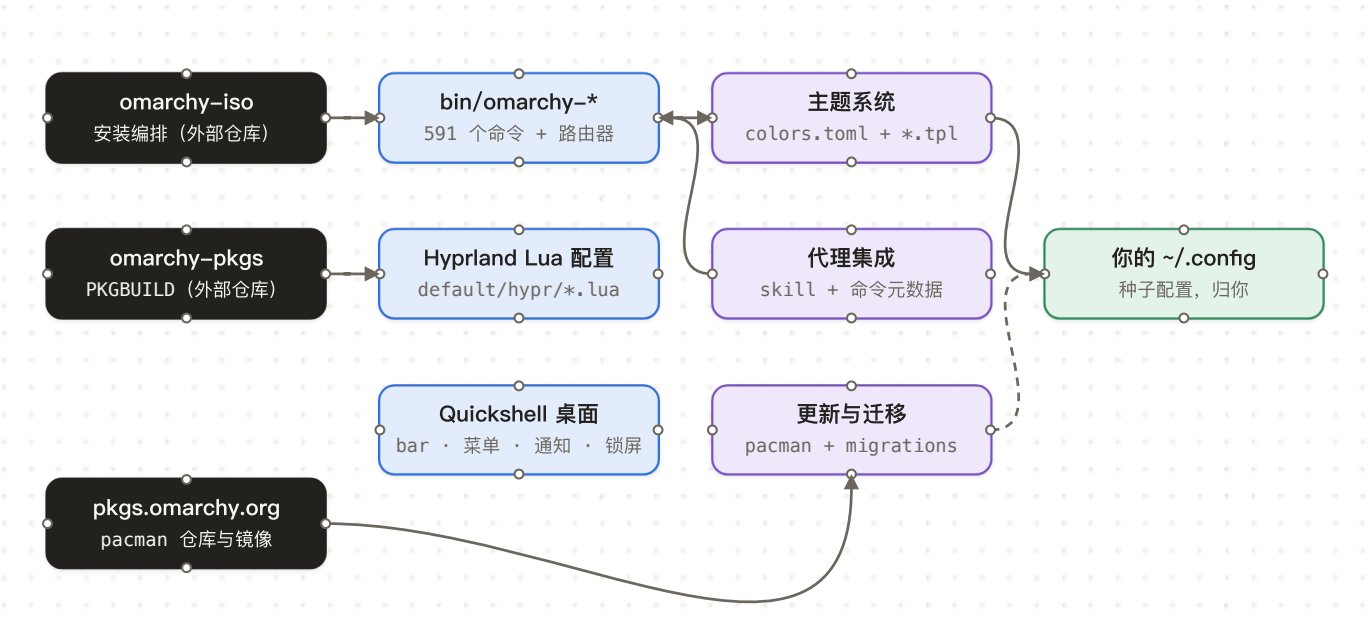
1. 全景：问题、目标、限制

1.1 没有它时的问题

环节	问题	Omarchy 的做法
装一台能用的 Linux 开发机	Arch + Hyprland 要手动拼几十个组件：状态栏、通知、锁屏、截图、输入法、主题、电源管理	一套“厨师选好的”（omakase）默认组合，装完就能用
键盘驱动的工作流	每个工具快捷键各自为政	统一的 Super 键“平面”规则，所有动作都能从菜单、快捷键、CLI 三条路触达
外观一致	终端、编辑器、浏览器、状态栏各有各的主题格式	一份 `colors.toml` + 模板，一条命令重新着色二十多个应用
滚动更新的风险	`pacman -Syu` 偶尔把系统弄坏	更新前自动快照、更新后跑迁移，坏了从启动菜单回滚
让 AI 代理帮你改系统	代理不知道系统有哪些能力、哪些文件能改	每个命令带机器可读元数据，并给各家代理装一份 Omarchy skill

1.2 一张总图

Omarchy 4.0 的组成与周边



黑色是不在本仓库的部分：ISO 安装器、打包脚本、pacman 仓库（见 `file-layout.md`）。蓝色是运行时的三大件，紫色是横跨它们的机制，绿色是用户自己的配置。

1.3 要求 / 目标 / 限制

维度	内容	后面哪一章解释
核心目标	装完即用、键盘驱动、好看、对 AI 代理友好	第 4、5、8 章
功能要求	一个命令换全系统主题；菜单 / 快捷键 / CLI 三条路调用同一实现；更新可回滚	第 3、6、7 章
平台范围	x86 与五种 ARM 平台（Apple Silicon、Snapdragon、NVIDIA N1x、GB10、通用 ARM），体验要一致	`AGENTS.md · Platforms`
性能约束	常驻 shell 空闲时**不得** spawn 任何进程；路由器、菜单、主题切换都有时间预算	第 5 章；`Background Work`
用户配置约束	更新不能覆盖用户改过的文件；默认值改进要能自动生效	第 2、4、7 章
安全约束	外部主题和插件可能执行代码；提权要区分有无终端	第 6、10 章
许可证	MIT	第 12 章

类比 · 一家定好菜单的餐馆

Omarchy 自称 omakase：厨师替你点好菜。`/usr/share/omarchy` 是**后厨**——菜谱和原料归厨房，每次进货（更新）都会整箱替换；`~/.config` 是**你的餐桌**——端上来以后怎么调味是你的事，厨房不会再碰；`~/.local/state/omarchy` 是**传菜口**——当前这一桌的配色、开关状态都在这里临时放着，由厨房生成，你不该伸手进去。看懂这三块地方各归谁，就看懂了 Omarchy 的大半设计。

2. 打包与安装

2.1 职责与难点

职责：把仓库里的东西放到目标机器的正确位置，并为每个用户准备好家目录。**难点**：有些文件必须在 `useradd -m` 之前就位（`/etc/skel`、limine、snapper 配置），有些只能在用户第一次图形登录时做（启用 user units、音频调校）；还要支持“给别人装机、首次开机再建用户”的预装镜像。

2.2 两个包

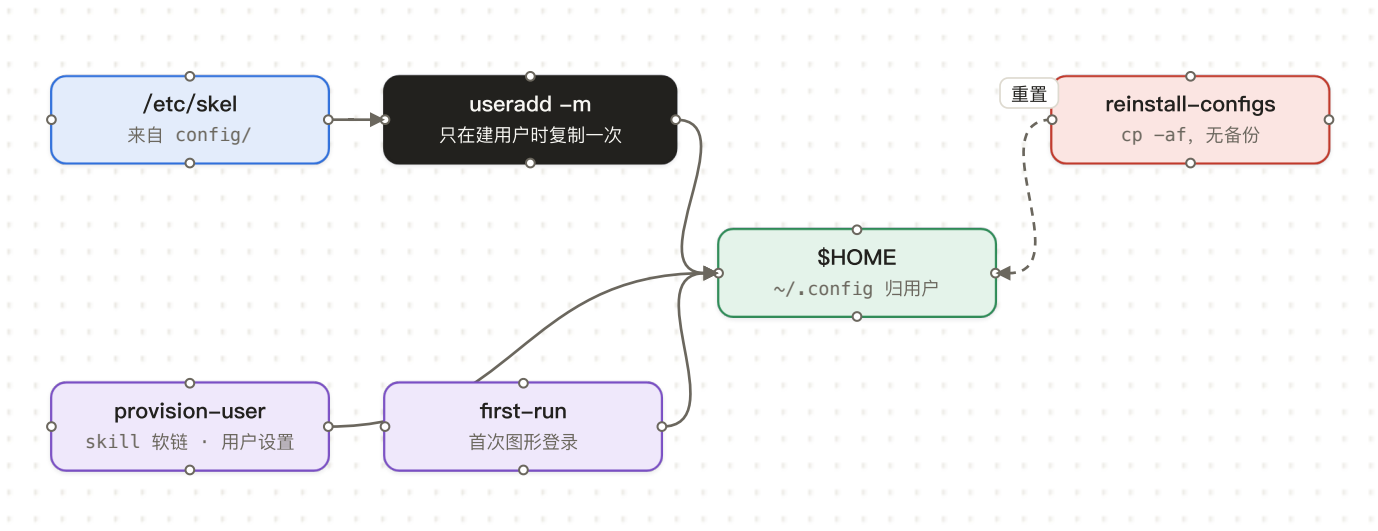
v4 从同一个仓库构建两个 Arch 包，PKGBUILD 在外部的 `omarchy-pkgs` 仓库（`file-layout.md`）：

包	内容	安装到
`omarchy`	`bin/`、`install/`、`migrations/`、`themes/`、`shell/`	`/usr/bin/omarchy-*`、`/usr/share/omarchy/`
`omarchy-settings`	`/etc/skel/**`（来自 `config/`）、`/etc` drop-in、字体、plymouth/sddm 主题、limine/snapper 配置	`/etc/**`、`/usr/share/omarchy/default/` 等

`OMARCHY_PATH` 由 `env-bootstrap` 设置：有 `/etc/omarchy.conf`（dev-link 写入）就读它，否则**强制**为 `/usr/share/omarchy`。按 `AGENTS.md` 的规定，运行时代码只能依赖这个变量，不准自己推导回退路径。

2.3 三层家目录

家目录是分三次填进去的



普通的 `omarchy update` 不会重新复制 `config/`（`file-layout.md · 三层`）。全量重置 `omarchy-reinstall-configs` 会直接覆盖，并且顺带重置 Limine 和 Plymouth。

2.4 安装流水线

本仓库**没有** `install.sh` 这类入口，安装由外部 ISO 编排（`install-scripts.md`）。仓库提供的是目标机上的三个命令：

阶段	命令（身份）	做什么
系统级	`omarchy-apply-system`（root，在 chroot 里）	依次 source `config/all.sh`（主题系统、浏览器策略、锁定策略、ssh、docker、snapper、防火墙…）→ 硬件叶子 `hardware/all.sh`（按厂商分组约 79 项）→ 登录 → post-install
用户级	`omarchy-provision-user`（用户）	把 agent skill 软链进 6 种代理的目录（`L87-101`），source `user/all.sh`，默认主题设为 Tokyo Night（`theme.sh`）
首次登录	`omarchy-provision-first-run`	启用 user units、平台分派 `setup-user`、音频调校、欢迎页

每个叶子脚本都通过 `run_logged` 执行并写进 `/var/log/omarchy-install.log`。包清单按 base → 架构 → 平台拼接（`omarchy-pkg-defaults`）：`base.packages` 有 164 个包。

容易误解的地方：`install/` 和 `migrations/` 下的脚本没有 shebang，是被 source 的，不能直接执行（`Style`）。

3. CLI 路由器：`omarchy <group> <cmd>`

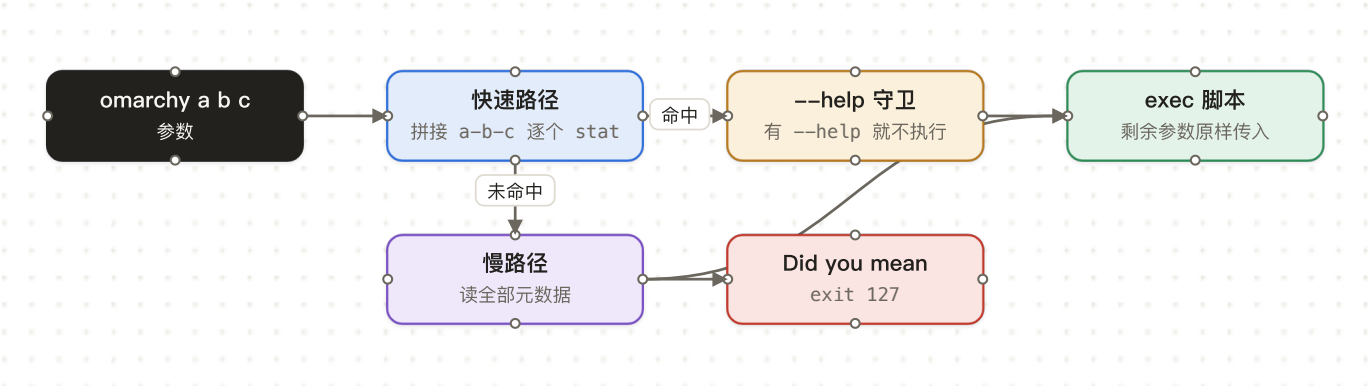
3.1 职责与难点

591 个命令要能用空格分词的方式调用（`omarchy theme set`），要能自省（给人看的帮助、给代理看的 JSON），而且**快**：快捷键、菜单、shell 都在频繁调用它。

3.2 原理：文件名就是路由

没有注册表：每个可执行的 `bin/omarchy-*` 就是一个命令，`omarchy-theme-set` 默认路由为 `omarchy theme set`（`cli-router.md`）。

路由器的分发流程



快速路径 `resolve_direct_route` 只做文件存在性检查、不读任何元数据；匹配不到（别名、元数据改过的路由）才进入 `resolve_route` 做全表最长前缀匹配。最后 `exec` 在 `L1073`。

3.3 契约：命令头部元数据

元数据写在文件开头的注释里，最多扫描 80 行（`METADATA_SCAN_LIMIT`），由 `register_command` 解析：

键	作用	例子
`summary` / `args` / `examples`	帮助文本，`examples` 用 `|` 分隔多个	`theme-set`
`alias`	额外的调用名	`omarchy up`、`omarchy screenshot`
`group` / `name`	覆盖默认路由	`omarchy share`
`hidden=true`	能调用但不列出（149 个命令）	`apply-system`
`requires-sudo=true`	标记需要提权	`omarchy-update`

顶层分组列表的唯一来源是 `GROUP_DESCRIPTIONS`（73 组）。`omarchy commands --json` 输出全部命令的结构化记录，是代理发现能力的入口（`自省与 --check`）。

容易误解的地方：参数里只要出现 `--help`（`--` 之后的除外）就只显示帮助、绝不执行（`remaining_has_help_flag`），所以给子命令传一个字面的 `--help` 需要放在 `--` 后面。

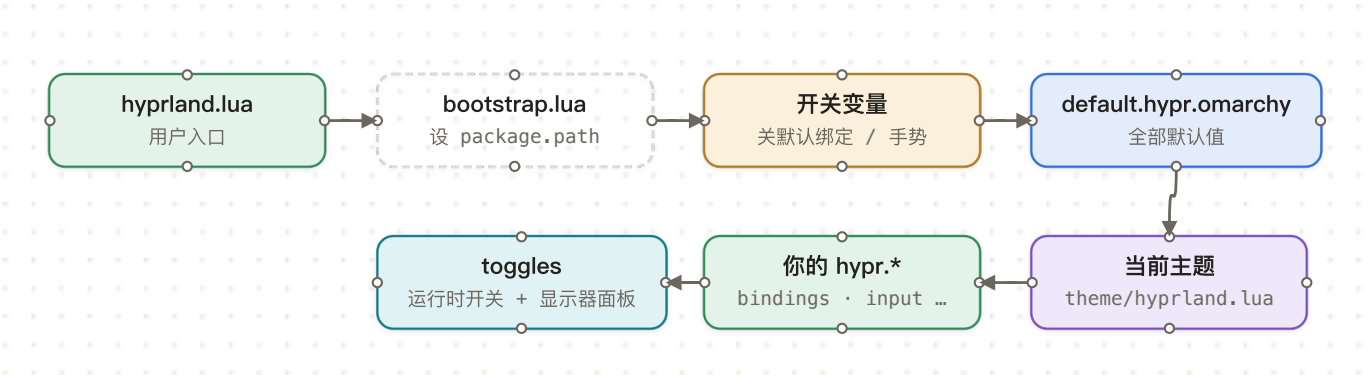
4. Hyprland：Lua 配置与快捷键体系

4.1 职责与难点

Hyprland 新版本支持用 Lua 写配置。Omarchy 用它解决一个老问题：**默认值要能随更新自动改进，而用户的修改又不能被覆盖**。v3 时代默认值直接复制进用户文件，更新后要么覆盖、要么永远拿不到新默认；Lua 的 `require` 让默认值留在 `/usr/share/omarchy`，用户文件只写差异。

4.2 加载顺序

~/.config/hypr/hyprland.lua 的加载顺序



后加载的覆盖先加载的。bootstrap 把 `package.path` 设成 state → config → OMARCHY_PATH（`bootstrap.lua`），默认值在 `omarchy.lua` 里加载，主题覆盖在 `L34`，用户模块在 `hyprland.lua L22–26`，最后是 `toggles.lua`——所以显示器面板的设置会赢过你写的 `monitors.lua`。

开关变量必须写在 `require("default.hypr.omarchy")` 之前：`omarchy_default_bindings = false` 关掉全部默认绑定，`omarchy_preinstalled_bindings = false` 只关预装应用和 Web App 的绑定，`omarchy_default_gestures = false` 关手势（`hyprland.lua L6–14`）。

4.3 快捷键的 DSL

所有默认绑定由 `bindings.lua` 从 `default/hypr/bindings/` 加载八个文件（tiling、applications、utilities、clipboard、media、dictation、input-method、jis）。绑定函数在 `helpers.lua`：

写法	翻译成	位置
`o.bind(keys, 描述, 动作)`	注册绑定，描述会出现在 `Super+K` 速查里	`L138`
`o.rebind(...)`	先 unbind 再 bind	`L154`
`{ menu = "theme" }`	打开某个菜单路由	`L97–98`
`{ webapp = url, focus = true }`	以 App 模式打开网页，已开则聚焦	`L113–117`
`{ tui = "btop" }`	在终端里启动一个 TUI	`L119–123`

性能细节：如果动作登记在 `default/omarchy/shortcuts` 里，`shell_dispatcher` 会把它变成 Hyprland 全局快捷键，按键直接交给常驻的 shell 进程，**不 fork 任何进程**；不在表里的才回退成执行命令。

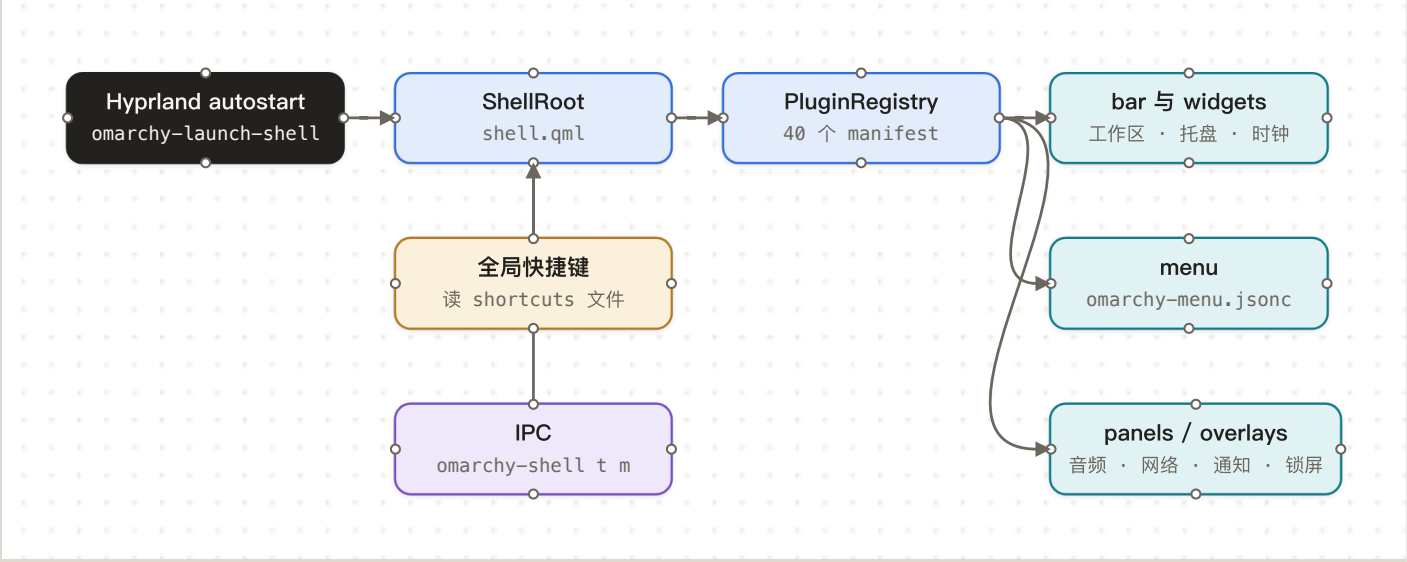
5. Quickshell 桌面与菜单

5.1 职责与难点

顶栏、菜单、通知、OSD、锁屏、剪贴板、emoji、dock、各种面板，在 v3 里是 waybar、mako、walker、hyprlock 等一堆独立程序；4.0 全部换成**一个**常驻的 Quickshell（QML）进程里的插件。难点是：它常驻、要省电，所以必须事件驱动；它是整个桌面，崩了或卡了就什么都不能用。

5.2 结构

一个进程承载整个桌面



启动见 `omarchy-launch-shell`，它刻意设置 `QS_DISABLE_FILE_WATCHER=1`，避免包升级到一半时热重载出错。根对象 `ShellRoot`，IPC 方法表从 `shell.qml L1796` 开始；插件种类与 manifest 字段见 `omarchy-shell.md`。

- shell 本身就是 freedesktop 通知守护进程（`notifications.md`）。
- 第三方插件 clone 到 `~/.config/omarchy/plugins/<id>/`，**没有沙箱**，拥有你用户的全部权限（`第三方插件`）。
- 空闲时 shell 不能 spawn 进程，由测试 `shell-idle-spawn-test.sh` 把关。

5.3 菜单：数据驱动

菜单是 `omarchy.menu` 插件，定义全部在一个 JSONC 文件 `omarchy-menu.jsonc` 里（约 436 个条目）。规则（`menu.md`）：

- key 用点号表示层级：`trigger.share.file` 是 `trigger.share` 的子项；有 `action` 的是动作，否则是子菜单。
- `when` / `checked` / `disabled` 是 bash 条件，打开菜单时批量放进 8 个并行子 shell 求值，先用上次结果、不等待（`Guards`）。
- `aliases` 是可直接 summon 的短名，快捷键用的就是它们（如 `theme` → `appearance.theme`，`L89`）。

- 用户的 `~/.config/omarchy/extensions/omarchy-menu.jsonc` 按 key 覆盖默认，新 id 追加到末尾（`mergeMenuSources`）。

> **类比 · 菜单是一张点菜单，命令是后厨的工位**
>
> 菜单上每一行只是“名字 + 去哪个工位”：`action` 里写的就是某个 `omarchy-*` 命令。快捷键、菜单、CLI 是三个点菜窗口，最后都落到同一个工位上。所以给系统加一个功能，正确的顺序是先在 `bin/` 做出工位（命令），再在菜单里加一行、需要的话绑一个快捷键——而不是把逻辑写进菜单或快捷键里。

6. 主题流水线

6.1 职责与难点

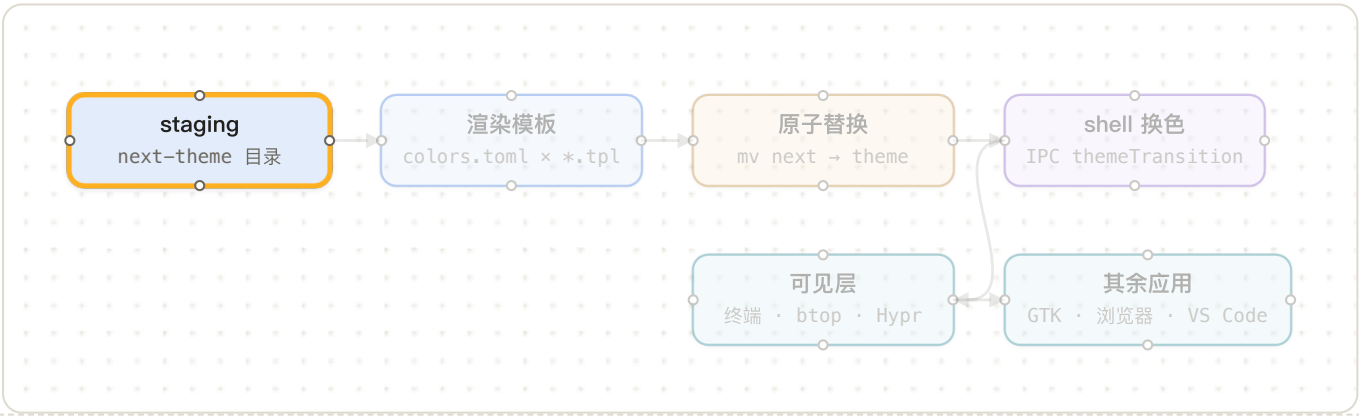
职责：`omarchy theme set <名字>` 之后，桌面边框、shell、四种终端、btop、Neovim、浏览器、GTK、VS Code、Obsidian、Slack，以及 Claude Code / OpenCode / Pi / Hermes 等代理的 TUI 全部换色。**难点**：二十多个应用接受主题的方式各不相同（重载、信号、文件 watch、策略文件）；切换要原子（不能出现一半新一半旧）；外部下载的主题可能夹带能执行的代码。

6.2 数据：调色板 + 模板

- 每个主题目录有一个语义调色板 `colors.toml`（accent、四级背景、四级前景、命名色及 bright_*，例如 `tokyo-night`），可选 `backgrounds/`、`preview.png`、`icons.theme`、手写的 `neovim.lua` / `vscode.json`、`shell.<section>.toml` 等（`theming.md`）。内置 23 个主题，其中 3 个浅色。
- `default/themed/` 下 23 个模板，占位符 `{{ background }}`、`{{ accent_rgb }}`、`{{ mix a b 15% }}` 等（`占位符`），由 `render_templates` 用一个 awk 进程全部渲染。主题目录里已有同名文件就跳过模板，所以**手写文件优先**；用户模板 `~/.config/omarchy/themed/*.tpl` 先于内置模板处理。

6.3 切换过程

▶ omarchy-theme-set 的五个阶段 1 / 5



1. 组装 staging

加锁（`flock`，`L398–399`），清空 `next-theme`，复制内置主题，再叠加 `~/.config/omarchy/themes/<名字>/`。如果用户主题目录里有 `.git`（即通过 `omarchy theme install` 装的），会**丢弃**所有 `.lua`、终端配置和 `vscode.json`，名单见 `INSTALLED_THEME_DENIED`——这些文件都能执行代码。

2. 渲染模板

`omarchy-theme-set-templates` 先用 `omarchy-theme-color --all` 解析调色板（含派生色），再把用户模板和内置模板渲染进 staging 目录。

3. 原子替换

`mv next-theme theme`（`L490`），再写 `theme.name`（`L501`）。读到当前主题的程序永远看到完整的一套。

4. shell 先换

调色板 base64 后经 `set_theme_background` 用 IPC `background themeTransition` 交给 shell，壁纸和颜色一起过渡（`applyPendingTheme`）。

5. 其余应用并行

释放锁后分两组并行：可见层（终端、btop、foot、tmux、Hyprland 重载，`L526–534`）和其余应用（GNOME、Qt、浏览器、VS Code、Obsidian、Slack、各代理、fcitx5、键盘 RGB，`L539–553`），最后执行用户 hook `theme-set`（`L601`）。

6.4 各应用怎么收到新主题

应用	机制	代码
Hyprland	`hyprctl reload`，重新 require 当前主题的 `hyprland.lua`	`restart-hyprctl`
Quickshell	IPC 推送调色板	`Color.qml`
Alacritty / Kitty / Ghostty	`touch` 触发 watch / `SIGUSR1` / `SIGUSR2`	`restart-terminal`
btop	`SIGUSR2`	`restart-btop`
Chromium 系浏览器	写 root 拥有的 managed policy `color.json`	`theme-set-browser`
GTK / GNOME	`gsettings` 设 color-scheme 和图标主题	`theme-set-gnome`

容易误解的地方：终端配置文件第一行只是 `import` 当前主题生成的文件（如 `alacritty.toml L1`）。终端的配色由主题管，不要在终端配置里直接改颜色——下次换主题就会被覆盖；要改就改主题或写一个 themed 模板。

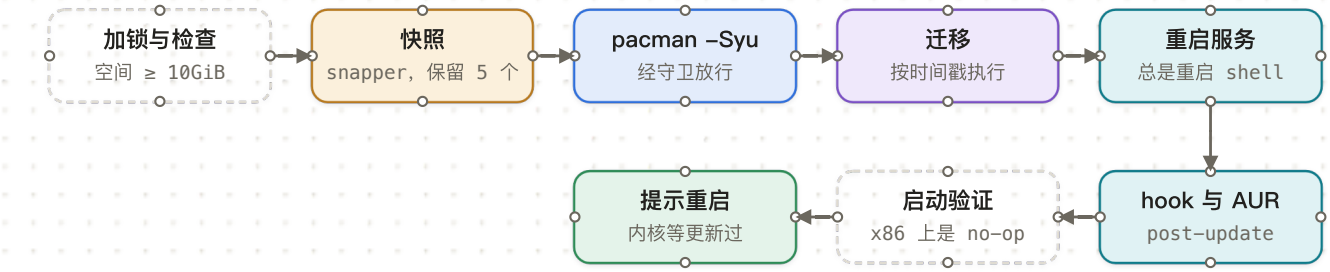
7. 更新、迁移与快照

7.1 职责与难点

滚动发行版的更新偶尔会坏；有些修复需要改用户已经存在的文件；更新时 Hyprland 和 shell 都在运行，不能在包替换到一半时重载。

7.2 `omarchy update` 的时间线

omarchy update 做了什么



全部 20 步在 `bin/omarchy-update`：快照 `L89`、包事务 `L101`、迁移 `L107`、hook `L121`。dev 通道才会额外 `git pull --ff-only`（`update-dev`）。

- **为什么直接 `sudo pacman -Syu` 会被拒绝**：ALPM 钩子 `00-omarchy-update-guard.hook` 在事务前运行 `pacman-guard`，只有带 `OMARCHY_UPDATE_PACMAN=1`（由 `omarchy update` 设置）或 `OMARCHY_ALLOW_DIRECT_PACMAN=1` 才放行——绕过会跳过快照和迁移。
- **pacman 跑在 PID 1 的 scope 里**（`systemd-run --scope`，`update-pacman`），这样 systemd 重新执行自己时不会把更新杀掉。
- `omarchy-settings` 升级期间，一对 ALPM 钩子先暂停、后恢复 Hyprland 的自动重载（`update-process.md`）。

7.3 迁移

- 格式：`migrations/<unix 时间戳>.sh`，没有 shebang，以用户身份执行，必须幂等；共 249 个。
- 运行器 `omarchy-migrate` 先等 pacman 锁，再按文件名顺序逐个 `bash -euo pipefail`，成功后在 `~/.local/state/omarchy/migrations/` 留标记（`L88–97`）。
- 改用户文件的迁移遵循“**只有当你的文件还是出厂原样才替换**”：例如 这个迁移 先比较 sha256。

> **类比 · 装修队只换你没动过的家具**
>
> 迁移像物业派来的装修队：每次进门前先看一眼清单（`~/.local/state/omarchy/migrations/` 里的标记），做过的活不再做；要换家具时先核对它还是不是交房时那一件（比较 sha256），你自己换过的就不碰。所以更新后某个默认改进没生效，往往正是因为你改过那个文件——这时候用 `omarchy-refresh-config` 拿一份新默认值，再把自己的改动并回去。

- 登录时只通知、不在后台静默执行（`migrate-notify`）。

7.4 快照与回滚

snapper 只给根分区配置，**只保留 5 个、没有定时快照**（`snapper/root`），每次更新前自动建一个；`limine-snapper-sync` 让快照出现在启动菜单里。Limine 菜单 1 秒后自动进入（`limine.conf`），开机时按任意键调出。

> **注意 · 快照不包括 /home**
>
> 回滚只恢复系统，不恢复 `~/.config`；迁移标记也在家目录里，所以回滚后再更新，**已经跑过的迁移不会重跑**。改 `~/.config/uwsm/env.d/` 写错导致登录后被踢回登录界面，快照也救不了——要进 TTY 把坏文件移走。

8. “Agentic”：AI 代理是怎么接进系统的

Omarchy 自称“agentic”，在源码里具体是五件事：

1. **一份给代理读的 skill**：`SKILL.md` 规定 `/usr/share/omarchy/` 只读，给出提权规则（有终端用 sudo、没有用 pkexec），教代理用 `omarchy commands --json` 发现能力、改配置前先备份、改完用 `hyprctl configerrors` 自检，refresh 前必须征得用户同意。
2. **装给六家代理**：`provision-user` 把它软链进 `~/.claude/skills`、`~/.codex/skills`、`~/.gemini/config/skills`、`~/.pi/agent/skills`、`~/.hermes/skills` 和通用的 `~/.agents/skills`。
3. **机器可读的 CLI**：每个命令的 summary/args/examples 都能用 JSON 导出（第 3 章）。
4. **代理工具链命令**：`omarchy-agent`（启动默认代理，`--local` 用本地模型）、多订阅账号切换、统一的 `~/.agents/AGENTS.md`、用量面板（约 23 个用量采集器，`agent-usage-collectors.md`）、本地模型（外部的 `omarchy-local-ai` 包，`local-models.md`）。
5. **主题延伸到代理**：Claude Code、Pi、Hermes、T3 Code 都有主题模板（`L542–545`）。

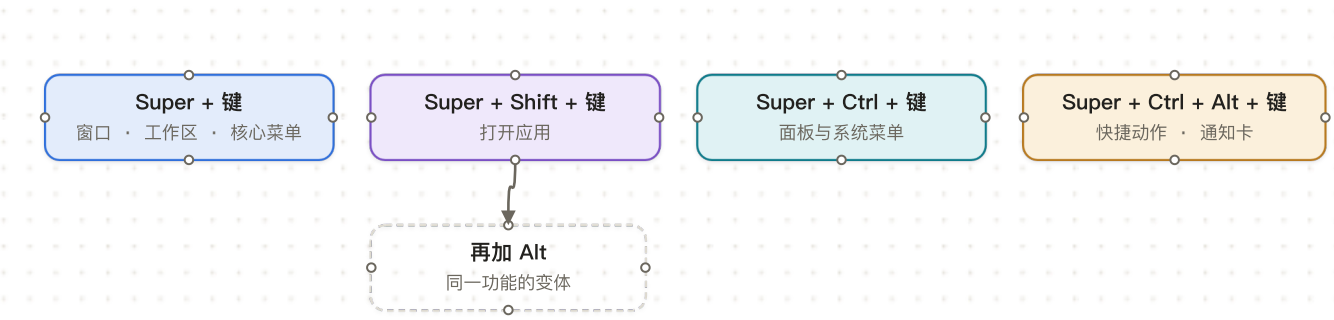
代理 CLI 本身（claude、codex、opencode 等 23 个）通过 mise 懒安装、每 4 小时自动更新（`omarchy-tools.toml`）。所以**不要**用 `claude update` 这类自带更新器，用 `omarchy update`。

9. 功能指南：用好这个 OS

9.1 心智模型：一切从键盘开始

手册的原话是“Everything in Omarchy happens via the keyboard — EVERYTHING!”。快捷键按 Super 键的“平面”排布（`manual/07`）：

快捷键的四个平面



规则是“按功能选平面，加 Alt 得到变体”：`Super+Shift+B` 浏览器，`Super+Shift+Alt+B` 隐私窗口（`applications.lua L6–7`）。`Super+K` 随时显示当前所有绑定。

最重要的三个键：`Super+Space` 打开菜单（可以直接打字过滤，`utilities.lua L1`）；`Super+K` 看快捷键速查；`Super+Escape` 系统菜单（挂起、重启、关机）。

9.2 快捷键速查（从源码整理）

窗口与工作区（`tiling.lua`）

按键	动作
`Super+W` / `Super+Q`	关闭窗口（应用真正退出）
`Super+方向键` · `Super+Shift+方向键`	移动焦点 · 与相邻窗口交换
`Super+1…0` · `Super+Shift+数字`	切换工作区 · 把窗口送过去（每个显示器各有 1–10）
`Super+Tab` / `Super+Ctrl+Tab`	下一个 / 上次用过的工作区
`Super+T` · `Super+F` · `Super+Alt+F`	平铺/浮动 · 全屏 · 全宽（保留顶栏）
`Super+J` · `Super+L`	切换分割方向 · 循环工作区布局 dwindle → scrolling → floating
`Super+` 反引号	Quake 下拉控制台（第一次打开自动启动默认 AI 代理，`qconsole.lua`）
`Super+S` · `Super+M`	scratchpad · 最小化到 dock（浮动工作区）
`Super+G`	窗口编组（标签页式）
`Super+-` / `Super+=`	调整宽度（加 Alt 细调、加 Ctrl 粗调）
`Ctrl+Alt+Delete`	**关闭所有窗口**——不是任务管理器！

应用（`applications.lua`）：`Super+Return` 终端（在当前终端的目录打开）、`Super+Alt+Return` tmux、`Super+Shift+B` 浏览器、`Super+Shift+F` 文件管理器、`Super+Shift+N` 编辑器、`Super+Shift+T` btop、`Super+Shift+/` 1Password、`Super+Shift+O` Obsidian、`Super+Shift+Ctrl+A` 默认 AI 代理，以及一组 Web App（`Super+Shift+A` ChatGPT、`Super+Shift+E` HEY 邮箱、`Super+Shift+Y` YouTube…）。

面板与工具（`utilities.lua`）

按键	动作
`Super+Ctrl+A / B / N / D / P`	音频 / 蓝牙 / 网络 / 显示 / 电源面板
`Super+Ctrl+V`	剪贴板历史（`clipboard.lua`）
`Super+Ctrl+E`	Emoji
`Super+Ctrl+C` · `Print`	截屏菜单 · 直接截图
`Alt+Print`	录屏开始/停止
`Super+Print` · `Super+Ctrl+Print`	取色 · OCR 识别屏幕文字
`Super+Ctrl+S`	LocalSend 分享（类似 AirDrop）
`Super+Ctrl+R`	设置提醒
`Super+Ctrl+O` · `Super+Ctrl+I`	Toggle 菜单 · 防休眠（Stay awake）
`Super+Ctrl+T` · `Super+Ctrl+W`	主题选择器 · 壁纸选择器
`Super+Ctrl+L`	锁屏
`Super+,` · `Super+Ctrl+Alt+,`	处理最新通知 · 勿扰
`Alt+Space` · 按住右 Alt	听写开关 · 按住说话（`dictation.lua`）

注意 · 手册与代码对不上的地方

`Super+M` 、 `Super+I` （切换输入法）、 `Super+Shift+D` （lazydocker，装了才有）都绑定了但 manual/07 的表里没有；manual/06 说有 22 个主题，实际 `themes/` 有 23 个；manual/13 说 Toggle 菜单里有 Suspend 和 Herdr 主题同步，实际菜单只有 idle、通知、夜灯、屏保、崩溃捕获（`L161–171`），另外两个只能用 CLI。以 `Super+K` 显示的为准。

9.3 按场景整理的内置功能

场景	功能	怎么用	容易漏掉的
复制粘贴	统一剪贴板	`Super+C/V` 在终端里自动换成 `Ctrl+Shift+C/V`	历史最多 500 条，存成**明文** `~/.local/state/omarchy/clipboard-history.json`（`L41`）；密码管理器复制的内容不入历史（`capture.sh`）
读截图里的字	OCR	`Super+Ctrl+Print`	设 `OMARCHY_OCR_LANGS=chi_sim+eng` 可识别中文（需要装 `tesseract-data-chi_sim`，`capture-text`）
截图 / 录屏	Omasnap / gpu-screen-recorder	`Print`、`Alt+Print`	录屏输出目录 `~/Videos` 必须事先存在（`L29`）；录完自动响度归一化
打字太慢	听写	按住右 Alt 说话	后端可选 Superwhisper / Voxtype / Hex（*Setup > Defaults > Dictation*）
定时提醒	提醒	`omarchy reminder 25 "check CI"`	底层是 systemd 用户定时器（`L194`），shell 重启不影响
演示或看构建	防休眠 / 勿扰	`Super+Ctrl+I` / `Super+Ctrl+Alt+,`	空闲时间在 `shell.json`：屏保 150 秒、锁屏 300 秒（`shell.json`）
常用网站	Web App	*Install > Web App*	支持非交互：`omarchy-webapp-install "Linear" https://linear.app <图标URL>`（`L4`）
常用 TUI	TUI App	*Install > TUI*	任何命令行工具都能变成有图标、有快捷键的“应用”（`L4`）
开发布局	tmux 布局函数	`tdl`、`tds`、`tsl 4 cx`	`tdl` = 编辑器 + AI + 终端（`fns/tmux`）；`ga <branch>` 在 `../<repo>--<branch>` 建 worktree（`worktrees`）
文件互传	LocalSend / Taildrop	`Super+Ctrl+S`	防火墙默认只开放 LocalSend 端口；跨网络用 `omarchy tailscale send`
Windows 软件	Windows 虚拟机	*Install > Windows*	基于 dockur/windows + RDP；*Remove > Windows* 会删掉整个虚拟盘
跑本地模型	本地 AI	`omarchy agent local use best`	Podman 运行，OpenAI 兼容接口 `127.0.0.1:11435`，不联网

9.4 shell 里的小工具

`ff`（fzf + bat 预览）、`eff`（用编辑器打开选中的文件）、`cd`（zoxide 增强）、`ls`（eza）、`n`（`nvim .`）、`t`（tmux）、`a`（在当前目录起默认代理）。别名 `c` / `cx` / `cy` 分别是 opencode、claude、codex 的**自动批准**模式（`aliases`）——方便，但意味着代理执行命令不再逐条确认。

9.5 给开发者的十个习惯

1. **一切先按 `Super+Space` 再打字**；用熟的菜单项在 `~/.config/hypr/bindings.lua` 里绑成快捷键（`{ menu = "<alias>" }`）。
2. **一个项目一个工作区**，代码工作区用 scrolling 布局（`Super+L`），布局按工作区记住、重启后保留。
3. **用 `Super+Ctrl+V` 剪贴板历史**代替来回切窗口复制。
4. **把 `Super+` 反引号当随叫随到的 AI 控制台**。
5. `tdl` / `tsl` + `ga` **worktree**：一键搭好编辑器 + 代理 + shell，并行任务各自隔离在 worktree 里。
6. **常用 SaaS 都做成 Web App 并绑快捷键**，带 `focus = true` 的是单实例。
7. **说比打快**：用听写写 commit message、PR 描述和给代理的 prompt。
8. **长任务挂提醒**（`omarchy reminder` 可以放在脚本末尾），看构建时开 Stay awake。
9. **TUI 都放在快捷键上**：btop、lazygit、lazydocker，自己的工具用 `omarchy-tui-install`。
10. **把定制交给代理，但保持可回滚**：代理已经装好了 Omarchy skill；先用 plan 模式；`~/.config` 纳入 git；给代理开免密 sudo 时只选 15 分钟或 1 小时。

9.6 从 Mac / Windows 过来

原来的	Omarchy
Spotlight / 开始菜单	`Super+Space`
Cmd+Tab / Alt+Tab	`Alt+Tab` 只在当前工作区内切窗口；跨应用用 `Super+数字`
虚拟桌面 / Spaces	每个显示器 10 个工作区
Cmd+Shift+4 / Win+Shift+S	`Print`，或 `Super+Ctrl+C` 截屏菜单
Win+V 剪贴板历史	`Super+Ctrl+V`
Live Text / 截图识字	`Super+Ctrl+Print`
AirDrop	`Super+Ctrl+S`（LocalSend）
控制中心	`Super+Ctrl+A / N / B / D / P` 各面板
Amphetamine / Caffeine	`Super+Ctrl+I`
活动监视器 / 任务管理器	`Super+Shift+T`（btop）；`Ctrl+Alt+Del` 在这里是关闭所有窗口
系统设置	菜单 *Setup*（直接打开对应配置文件）
软件更新	`omarchy update`（自动先做快照）
Time Machine（系统部分）	每次更新前的 snapper 快照（不含家目录）

10. 定制与维护

10.1 我想改 X → 改哪个文件

想改什么	改哪里	命令 / 菜单	更新会不会覆盖
快捷键	`~/.config/hypr/bindings.lua`（`o.bind` / `o.rebind` / `hl.unbind`）	*Setup > Keyboard > Keybindings*	不会
键盘布局、触控板	`~/.config/hypr/input.lua`	*Setup > Input*	不会
间隙、圆角、动画、默认布局	`~/.config/hypr/looknfeel.lua`	*Setup > Config*	不会
开机自启	`~/.config/hypr/autostart.lua`（`o.launch_on_start`）	—	不会
显示器缩放	显示器面板（存在 state 里，优先于 `monitors.lua`）	`Super+/`	不会
顶栏布局、时钟格式	`~/.config/omarchy/shell.json`	`omarchy bar set …`，`omarchy bar defaults` 恢复	不会；但有了自己的 `shell.json` 后**不再合并**新的默认组件
改内置顶栏组件	`omarchy plugin clone omarchy.clock`	克隆后对原 id 的调用会路由到你的副本	不会
菜单加项	`~/.config/omarchy/extensions/omarchy-menu.jsonc`	—	不会
自建或微调主题	`~/.config/omarchy/themes/<名字>/`	与内置同名时叠加在内置主题之上	不会；但 `omarchy theme install` 遇到同名目录会先 `rm -rf`（L52）
给其他应用上主题色	`~/.config/omarchy/themed/<文件>.tpl`	换一次主题生效	不会
环境变量（截图目录等）	`~/.config/uwsm/env.d/<文件>`	重新登录	不会；**写错会登录失败**
中文字形	fontconfig	`omarchy font cjk sc`（默认是日文字形）	不会
中文输入法	fcitx5	`omarchy setup input pinyin`，`Super+I` 切换	不会

想改什么	改哪里	命令 / 菜单	更新会不会覆盖
自定义 pacman 仓库	`~/.config/omarchy/hooks/pre-refresh-pacman.d/`	直接改 `/etc/pacman.conf` 会被整文件替换	放 hook 里才持久
更新后自动做事	`~/.config/omarchy/hooks/post-update.d/`	`omarchy hook install post-update <文件>`	不会

hook 是用 `bash "$hook"` 执行的（`omarchy-hook`），所以不需要可执行位，但 shebang 会被忽略——写 Python 要用 bash 包一层。

10.2 会覆盖你配置的命令

命令	范围	有没有备份
`omarchy-refresh-config <文件>`	单个文件	有，`.bak.<时间戳>`，内容没变则删除（`L20–39`）
Update > Config > Hyprland	7 个 hypr 文件	有
`omarchy refresh shell`	`shell.json`	有
`omarchy-font-set`	原地 `sed -i` 改终端字体行	无（并且会把 Stow 软链变成普通文件）
`omarchy theme install`	同名主题目录	无，先 `rm -rf`
`omarchy reinstall configs`	所有种子配置 + Limine + Plymouth	**无**
`omarchy reinstall`	包重装 + 回到 stable + 重置配置	**无**

10.3 维护习惯

1. **只用** `omarchy update`，不要直接 `pacman -Syu` / `yay -Syu`；保持 stable 通道（它的 Arch 镜像比上游晚一个月，换来的是稳定）。
2. 更新输出里出现“boot files did not pass verification”时**不要重启**（`L149–154`）。
3. 出现 `.bak.<时间戳>` 文件时 diff 一下，把自己的改动并回去。
4. 高风险改动（env.d、内核参数、换通道、大量 AUR 包）之前：`omarchy-snapshot create`，并把 dotfiles commit 一次——快照只有 5 个名额，也不含家目录。
5. 用 git 管理 `~/.config/hypr/`、`~/.config/omarchy/`、`~/.config/uwsm/env.d/`、`~/.bashrc`；不要纳入 `~/.local/state/omarchy/`。用 Stow 软链的话，注意 `shell.json` 和终端配置会被命令原子重写成普通文件，定期 `stow --adopt` 回收。

6. 出问题时从轻到重：*Update > Hardware* 重启单个硬件 → `omarchy-restart-shell` → 单文件 refresh → 启动菜单选快照回滚 → `omarchy-debug` 去 Discord 求助 → `omarchy reinstall`。

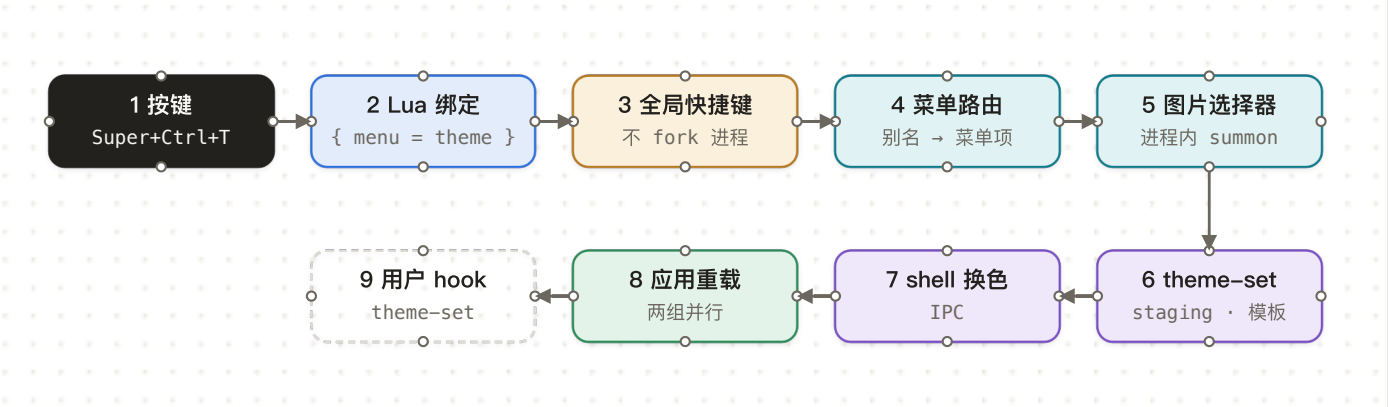
要点 · 安全默认值

- 防火墙拒绝所有入站，只开放 LocalSend 端口；SSH 默认关闭，在 *Setup > Security > SSHD* 打开并限速（`firewall.sh`）。
- sudo 10 分钟超时；连续输错 10 次锁定 2 分钟，TTY 里 `faillock --reset` 解锁。
- 免密 sudo 的限时授权就是一个带 `NOTAFTER` 的 sudoers 文件，开机时被 tmpfiles 清掉（`passwordless-sudo.md`）。
- 默认 LUKS 全盘加密（分区代码在外部 ISO 里）。

11. 端到端：按下 `Super+Ctrl+T` 到全系统换色

挑一条最能串起各组件的路径：用快捷键打开主题选择器，选一个主题。

一次换主题经过的组件



1–5 全在 shell 进程里完成，没有 spawn 任何进程；从第 6 步开始才执行 `omarchy-theme-set` 这个 bash 命令。外部的 OWE 壁纸引擎负责视频壁纸和开场动画。

1. 绑定：`utilities.lua L17` `o.bind("SUPER + CTRL + T", "Theme menu", { menu = "theme" })`。
2. `command_from` 看到 `menu`，因为 `shortcuts` 文件登记了 `menu theme`，`shell_dispatcher` 生成 `hl.dsp.global("omarchy:menu.theme")`。
3. shell 里由 `parseShortcuts` 注册的 GlobalShortcut 被触发，调用 `runShortcut` → `shell.toggle("omarchy.menu", {menu:"theme"})`。
4. 菜单 `resolveRoute` 把别名 `theme` 解析到 `appearance.theme`（`L89`），它是一个动作。
5. `summonAction` 认出动作是 `omarchy-shell shell summon omarchy.image-picker …`，于是在进程内打开图片选择器，用户确认后执行 `omarchy-theme-set <名字>`（`applySelected`）。CLI 路径 `omarchy theme set "Tokyo Night"` 从这里汇合：路由器快速路径找到 `omarchy-theme-set`，名字被规范化成 `tokyo-night`（`L379`）。
6. staging、模板、原子替换（第 6 章）。
7. shell 经 IPC 收到新调色板，顶栏、菜单、通知立即换色。
8. 终端、btop、Hyprland 先重载，GNOME、浏览器、VS Code 等随后并行。
9. 用户的 `~/.config/omarchy/hooks/theme-set*` 被执行；可选的跨机器同步（Herdr）默认关闭。

12. 开源许可证

部分	许可证	证据
仓库代码与配置	MIT，版权人 David Heinemeier Hansson	`LICENSE`、`README`
ALSA UCM2 音频配置	BSD 3-Clause（ALSA 项目）	`default/alsa/ucm2/` 下的 `LICENSE.*`
图标字体 `omarchy.ttf`	仓库只写了各字形的来源，没写许可证；其中是各公司的品牌标志	`fonts/omarchy/README.md`
主题壁纸	仓库没有来源或授权说明	`themes/*/backgrounds/`
外部组件	各自的许可证：Hyprland、Quickshell、Arch 包、omarchy-iso / omarchy-pkgs / omarchy-mac 等	不在本仓库

大白话：MIT 允许使用、修改、再分发甚至商用，只要保留版权声明和许可声明，作者不提供担保。

场景	可以吗	要做什么
自己用、改自己的 `~/.config`	可以	无
给上游提 PR	可以	遵守 AGENTS.md 的规则；仓库里没有 CLA
公开 fork / 做衍生发行版	可以	保留 LICENSE；建议换名换 logo——仓库没有商标政策，“Omarchy”名称和 logo 的立场需要另外确认
打包分发壁纸或图标字体	需要谨慎	壁纸没有授权说明，图标字体里是第三方品牌标志，MIT 不覆盖它们

历史（用 git 历史核实）：仓库从 2025-06-01 开始（第一个提交“Initial basics”，第二个是“Match omakub better”——它源自 DHH 早先给 Ubuntu 做的 Omakub）；LICENSE 在 2025-06-30 加入，从一开始就是 MIT，之后只补上了版权人，从未更换。到固定的 commit 为止共 7709 个提交、649 位贡献者，DHH 约占 65%。

> 以上不是法律意见。

13. 贡献与定制

13.1 开发环境

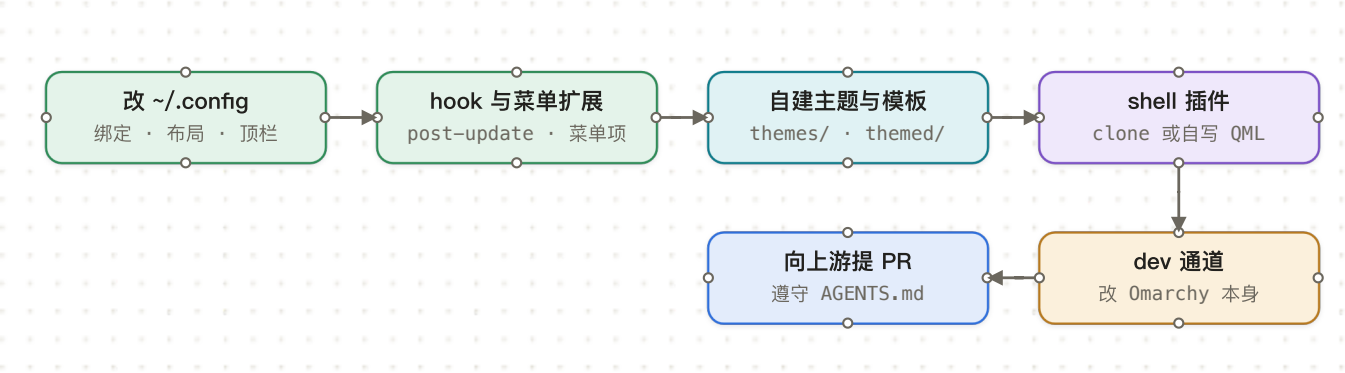
- 想改 Omarchy 本身，切到 dev 通道：`omarchy-channel-set dev` 会把 `OMARCHY_PATH` 链接到 `~/omarchy` 的 git checkout（`channel-set`），之后 `omarchy update` 会对它 `git pull --ff-only`。
- 测试入口：`./test/all`（CLI + shell 测试，不含图形验收）、`./test/cli`、`./test/shell`；只跑改动相关的文件，并把输出写进日志文件再读（`AGENTS.md · Tests`）。图形验收测试在一次性 VM 里通过同级的 `omarchy-iso` 仓库运行（`acceptance-tests.md`）。
- 做不同类型的修改前先读 `agents/skills/` 下对应的指南：命令元数据、安装脚本、mise 工具、shell 开发、动画、视觉验证、迁移。

13.2 review 规则（摘自 AGENTS.md）

- 所有命令以 `omarchy-` 开头，按前缀分组；shebang 必须是 `#!/bin/bash`；用 `[[ ]]` 和 `(( ))`；两空格缩进。
- 用 helper 而不是裸命令：`omarchy-pkg-add` / `omarchy-pkg-drop`、`omarchy-notification-send`、`omarchy-hw-*`（不要自己读 `uname -m`）。
- 平台缺包或出 bug：先推动上游修，平台专属替代方案是最后手段。
- 新快捷键按平面规则选键，不挪动已有绑定；新菜单项不加 `aliases`。
- 测试必须在不带这次修改时失败；声明式改动（一行菜单、一个配置值）不写测试；迁移不写测试。
- 有视觉效果的改动必须在运行中的 UI 里验证。
- commit 要原子化。

13.3 定制路线

从轻到重的定制方式



前三种都不碰 `/usr/share/omarchy`，更新安全；插件没有沙箱，装别人的插件等于运行别人的代码；dev 通道下系统直接跑你的 checkout，改坏了要自己修。

方式	代价	依赖
改 `~/.config`	几乎为零	理解第 4 章的加载顺序
hook / 菜单扩展	低	hook 用 bash 执行
自建主题、themed 模板	低	第 6 章的占位符语法
shell 插件	中：要写 QML，遵守“空闲不 spawn 进程”	`omarchy-shell.md`、`plugins/README.md`
dev 通道 / 提 PR	高：系统跑在你的 checkout 上	测试与 review 规则

13.4 仓库里没有的东西

- **安装器与分区、LUKS 加密**：在外部的 `omarchy-iso` 仓库。
- **打包脚本**：`omarchy-pkgs` 仓库；pacman 仓库在 `pkgs.omarchy.org`（`pacman-stable.conf`）。
- **Apple Silicon 支持**：`omarchy-mac` / `omarchy-mac-boot` 包，经 `lifecycle-dispatch` 调用，其他平台对应的操作都是 no-op。
- **独立程序**：Omasnap（截图）、OWE（壁纸引擎）、`linux-omarchy` 内核、`omarchy-nvim`（Neovim 配置，本仓库没有 nvim 目录）、`omarchy-local-ai`、Omawrite / Omacalc / Signed 等小工具。
- **上游**：Hyprland 的 Lua 配置 API、Quickshell。

14. 附录：源码索引

主题	入口
贡献规则	`AGENTS.md`
文件布局与安装位置	`docs/file-layout.md`
CLI 路由器	`bin/omarchy`、`cli-router.md`
命令元数据规范	`command-metadata.md`
系统级安装	`omarchy-apply-system`、`install/config/all.sh`
用户级安装	`omarchy-provision-user`、`install/user/all.sh`
包清单	`omarchy-base.packages`
Hyprland 默认配置	`default/hypr/omarchy.lua`、`helpers.lua`
用户 Hyprland 入口	`config/hypr/hyprland.lua`、`bindings.lua`
默认快捷键	`default/hypr/bindings/`
shell 根与 IPC	`shell/shell.qml`、`omarchy-shell.md`
菜单	`omarchy-menu.jsonc`、`Menu.qml`、`menu.md`
主题	`omarchy-theme-set`、`theme-set-templates`、`default/themed/`、`theming.md`
更新	`omarchy-update`、`update-process.md`
迁移	`omarchy-migrate`、`migrations/`
快照与启动	`omarchy-snapshot`、`snapper/root`、`limine.conf`
平台分派	`omarchy-hw-platform`、`lifecycle-dispatch`
代理 skill	`SKILL.md`、`omarchy-agent`
用户手册	`manual/`